

External Project Report on Computer Organization and Architecture (EET 2211)

Design A 16-bit Calculator (Arithmetic,
Logical And Number System Conversion)
Using 8086 Assembly Language



Submitted by

AMULYA SHRIVASTAVA	Reg. No.:	2341019456
ANURAG PRADHAN	Reg. No.:	2341016359
AMAAN AHMED	Reg. No.:	2341001055
ISHAN ARYADIT	Reg. No.:	2341001110

B. Tech. CSE 4th Semester (Section – 23412F3)

INSTITUTE OF TECHNICAL EDUCATION AND RESEARCH
(FACULTY OF ENGINEERING)

SIKSHA 'O' ANUSANDHAN (DEEMED TO BE UNIVERSITY), BHUBANESWAR,
ODISHA

Declaration

We, The Undersigned Students Of B. Tech. Of CSE Department Hereby Declare That We Own The Full Responsibility For The Information, Results Etc. Provided In This Project Titled “Design An 16-Bit Calculator (Arithmetic, Logical And Number System Conversion) Using 8086 Assembly Language” Submitted To Siksha ‘O’ Anusandhan Deemed To Be University, Bhubaneswar For The Partial Fulfillment Of The Subject Computer Organization And Architecture (EET 2211). We Have Taken Care In All Respect To Honor The Intellectual Property Right And Have Acknowledged The Contribution Of Others For Using Them In Academic Purpose And Further Declare That In Case Of Any Violation Of Intellectual Property Right Or Copyright We, As The Candidate(s), Will Be Fully Responsible For The Same.

(AMULYA SHRIVASTAVA)

Registration No.:

2341019456

(ANURAG PRADHAN)

Registration No.:

2341016359

(AMAAN AHMED)

Registration No.:

2341001055

(ISHAN ARYADIT)

Registration No.:

2341001110

DATE: 20/05/2025

PLACE: I.T.E.R, SOA (BHUBANESWAR)

Abstract

This Project Presents The Design And Implementation Of An 16-Bit Calculator Using 8086 Assembly Language, Capable Of Performing Arithmetic Operations, Logical Functions, And Number System Conversions. The Calculator Supports Basic Arithmetic Operations Including Addition, Subtraction, Multiplication, Division, And Remainder Calculation, While Also Incorporating Logical Operations Such As AND, OR, XOR, And NOT. In Addition To These Functions, The Calculator Effectively Converts Numerical Results Into Binary, Decimal And Hexadecimal Number Systems For Output Display. Developed Specifically For The Intel 8086 Microprocessor Architecture, The Program Utilizes Direct Register Manipulation And DOS Interrupts To Interact With The User Through Keyboard Input And Screen Output. The Implementation Ensures Robust Error Handling, Including Division By Zero Detection, To Maintain Program Stability. This Project Provides A Practical Understanding Of Low-Level Programming Concepts, Bitwise Operations, And Real-Time User Interaction In Assembly Language. It Acts As An Educational Tool For Those Interested In Exploring Computer Architecture, Embedded Systems, And The Inner Workings Of Computational Logic At The Hardware Level.

Contents

Serial No.	Chapter No.	Title of the Chapter	Page No.
1.	1	Introduction	01
2.	2	Problem Statement	02
3.	3	Methodology	03
4.	4	Implementation	04-10
5.	5	Results And Interpretation	11-13
6.	6	Conclusion	14
7.		References	15

1. Introduction

The 8086 Microprocessor, Developed By Intel, Is A Powerful 16-Bit Processor That Laid The Foundation For X86 Architecture. Programming In 8086 Assembly Language Provides Valuable Insight Into Hardware-Level Computation, Precise Register Manipulation, And Direct Memory Access. This Project Focuses On Designing An 16-Bit Calculator Using 8086 Assembly Language, Supporting Arithmetic Operations Such As Addition, Subtraction, Multiplication, Division, And Modulus; Logical Operations Including AND, OR, XOR, And NOT; Along With Number System Conversions To Binary, Decimal, And Hexadecimal Formats. The Calculator Accepts Two 8-Bit Hexadecimal Inputs From The Keyboard And Uses DOS Interrupts For Input And Output Handling, Enabling Real-Time User Interaction. Each Operation Is Performed Using Low-Level Instructions With Proper Register Usage, Error Handling For Cases Like Division By Zero, And Bitwise Manipulation For Logical Results. Conversion Routines Are Carefully Implemented To Accurately Display Binary And Decimal Equivalents Of The Result. Through This Implementation, The Project Demonstrates The Efficiency And Transparency Of Low-Level Programming In Performing Fundamental Computational Tasks, Making It An Excellent Educational Tool For Understanding Microprocessor Architecture, Instruction-Level Execution, And Assembly Language Programming.

2. Problem Statement

The Objective Is To Design An 16-Bit Calculator Using 8086 Assembly Language That Performs Arithmetic Operations Such As Addition, Subtraction, Multiplication, Division, And Remainder Calculation, Along With Logical Operations Including AND, OR, XOR, And NOT. The Calculator Also Supports Number System Conversions, Displaying The Result In Binary, Decimal And Hexadecimal Formats. It Accepts Two 8-Bit Hexadecimal Digits As Input Through The Keyboard, Processes Them Using 8086 Instructions, And Displays The Output Using DOS Interrupts. Division By Zero Is Appropriately Handled With A Custom Error Message. This Project Demonstrates Practical Implementation Of Low-Level Programming Concepts, Including Register Operations, Bitwise Manipulation, And Direct Interaction With System Interrupts. It Enhances Understanding Of Microprocessor Functionality, Efficient Use Of CPU Registers, And Real-Time Execution In A DOS-Based Environment.

3. Methodology

Define Requirements: Identify And Categorize Required Operations— Arithmetic, Logical, And Number System Conversions.

Setup Environment: Use An Emulator Or Assembler Like MASM/TASM To Write And Test 8086 Assembly Code.

Input Handling: Capture 16-Bit User Input From The Keyboard Using DOS Interrupts (INT 21h).

Data Validation: Ensure Inputs Are Valid 16-Bit Values And Fall Within Acceptable Number System Formats.

Arithmetic Operations: Implement Addition, Subtraction, Multiplication, And Division Using 8086 Instructions.

Logical Operations: Program Bitwise Operations Such As AND, OR, XOR, And NOT.

Conversion Functions: Develop Routines To Convert Numbers Between Binary, Decimal And Hexadecimal.

Result Display: Use INT 21h Functions To Display Results Clearly On The Screen.

User Interface Loop: Create A Looped Menu System For Continuous User Interaction And Operation Selection.

Continuation Loop: Create A Looped Menu System Which Makes The User To Input Choices To Exit/Halt The Program Or Re-Run The Loop.

Testing And Debugging: Test Each Function Separately, Debug Errors, And Ensure Accurate Computation And Display.

4. Implementation

Here Is A Basic **Implementation** Of An **16-bit Calculator** In **8086 Assembly** Using **EMU8086**. It Includes A Simple **Menu System**, And Performs **Arithmetic Operations, Logical Operations, And Number System Conversions** For 16-bit Inputs.

The Assembly Code Is As Follows:

```
.model small
.stack 100h

.data
    msg1      db 13, 10, 'Enter First Hex Number (0-FFFF): $'
    msg2      db 13, 10, 'Enter Second Hex Number (0-FFFF): $'
    msgAdd    db 13, 10, 10, 'Addition: $'
    msgSub    db 13, 10, 'Subtraction: $'
    msgMul    db 13, 10, 'Multiplication: $'
    msgDiv    db 13, 10, 'Division (Quotient): $'
    msgRem    db 13, 10, 'Division (Remainder): $'
    msgAnd    db 13, 10, 'AND: $'
    msgOr     db 13, 10, 'OR: $'
    msgXor    db 13, 10, 'XOR: $'
    msgNot1   db 13, 10, 'NOT Of First: $'
    msgNot2   db 13, 10, 'NOT Of Second: $'
    msgBin1   db 13, 10, 10, 'First In Binary: $'
    msgBin2   db 13, 10, 'Second In Binary: $'
    msgDec1   db 13, 10, 'First In Decimal: $'
    msgDec2   db 13, 10, 'Second In Decimal: $'
    newline   db 13, 10, '$'
    hexChars  db '0123456789ABCDEF'
    divByZero db 13, 10, 'Division By Zero!', 13, 10, '$'
    promptRepeat db 13, 10, 10, 'Enter 0 To Exit Or 1 To Repeat: $'

.code
main:
    mov ax, @data
    mov ds, ax

program_loop:
    ; Print Message 1 And Read First Hex Number (16-bit)
    lea dx, msg1
    mov ah, 09h
    int 21h
    call ReadHex16 ; Now Reads 16-bit Value
    mov bx, ax    ; Store First Number In BX

    ; Print Message 2 And Read Second Hex Number (16-bit)
    lea dx, msg2
    mov ah, 09h
```



```

int 21h
call ReadHex16
mov cx, ax    ; Store Second Number In CX

; ----- ARITHMETIC OPERATIONS (16-bit) -----
; Addition
lea dx, msgAdd
mov ah, 09h
int 21h
mov ax, bx
add ax, cx
call PrintHex16
; Subtraction
lea dx, msgSub
mov ah, 09h
int 21h
mov ax, bx
sub ax, cx
call PrintHex16
; Multiplication (16-bit 16-bit = 32-bit result)
lea dx, msgMul
mov ah, 09h
int 21h
mov ax, bx
mul cx        ; DX:AX = AX * CX
call PrintHex16 ; Print lower 16 bits (AX)
; Division (16-bit / 16-bit)
lea dx, msgDiv
mov ah, 09h
int 21h
cmp cx, 0
je divisionError
mov dx, 0     ; Clear Upper Part For Division
mov ax, bx    ; Dividend In AX
div cx        ; AX = Quotient, DX = Remainder
push dx       ; Save Remainder

; Print Quotient
call PrintHex16

; Print Remainder
lea dx, msgRem
mov ah, 09h
int 21h
pop ax        ; Restore Remainder
call PrintHex16

jmp afterDivision ; Skip Error Label

```

```

divisionError:
lea dx, divByZero
mov ah, 09h

```

int 21h

afterDivision:

; ----- LOGICAL OPERATIONS (16-bit) -----

; AND

lea dx, msgAnd

mov ah, 09h

int 21h

mov ax, bx

and ax, cx

call PrintHex16

; OR

lea dx, msgOr

mov ah, 09h

int 21h

mov ax, bx

or ax, cx

call PrintHex16

; XOR

lea dx, msgXor

mov ah, 09h

int 21h

mov ax, bx

xor ax, cx

call PrintHex16

; NOT (First Number)

lea dx, msgNot1

mov ah, 09h

int 21h

mov ax, bx

not ax

call PrintHex16

; NOT (Second Number)

lea dx, msgNot2

mov ah, 09h

int 21h

mov ax, cx

not ax

call PrintHex16

; ----- NUMBER SYSTEM CONVERSIONS (16-bit) -----

; First Number In Binary

lea dx, msgBin1

mov ah, 09h

int 21h

mov ax, bx

call PrintBinary16

; Second Number In Binary

lea dx, msgBin2

mov ah, 09h

int 21h

mov ax, cx

call PrintBinary16

; First Number In Decimal

lea dx, msgDec1

mov ah, 09h

int 21h

mov ax, bx

call PrintDecimal16

; Second Number In Decimal

lea dx, msgDec2

mov ah, 09h

int 21h

mov ax, cx

call PrintDecimal16

; Ask User If They Want To Repeat

lea dx, promptRepeat

mov ah, 09h

int 21h

; Read User Choice

mov ah, 01h

int 21h

cmp al, '1'

je program_loop ; Repeat If Input Is '1'

; Exit Program If Input Is Not '1'

mov ah, 4Ch

int 21h

; Subroutine: ReadHex16

; Reads A 4-digit Hex Number (0-FFFF) And Returns In AX

ReadHex16:

push bx

push cx

push dx

mov cx, 4 ; Read 4 hex digits (16-bit)

xor bx, bx ; Clear BX For Result

ReadNextDigit:

mov ah, 01h

int 21h ; Input Character In AL

; Convert ASCII To Hex Value

```

cmp al, '0'
jb InvalidChar16
cmp al, '9'
jbe Digit16
cmp al, 'A'
jb InvalidChar16
cmp al, 'F'
jbe AlphaUpper16
cmp al, 'a'
jb InvalidChar16
cmp al, 'f'
jbe AlphaLower16

```

InvalidChar16:

```

mov al, 0 ; Default To 0 On Invalid Input
jmp SkipShift16

```

Digit16:

```

sub al, '0' ; Convert '0'-'9' To 0-9
jmp ShiftAndAdd16

```

AlphaUpper16:

```

sub al, 'A' ; Convert 'A'-'F' To 10-15
add al, 10
jmp ShiftAndAdd16

```

AlphaLower16:

```

sub al, 'a' ; Convert 'a'-'f' To 10-15
add al, 10

```

ShiftAndAdd16:

```

shl bx, 4 ; Shift Left By 4 bits (For Next Digit)
or bl, al ; Add The New Digit

```

SkipShift16:

```

loop ReadNextDigit

```

```

mov ax, bx ; Return Result In AX

```

```

pop dx
pop cx
pop bx
ret

```

; Subroutine: PrintHex16

; Prints AX (16-bit value) In 4-digit HEX

PrintHex16:

```

push ax
push bx
push cx
push dx

```

```
mov cx, ax    ; Save Original Value
lea bx, hexChars ; Point To Hex Character Table
```

```
; Print High Byte First
mov al, ch    ; High byte
shr al, 4
and al, 0Fh
xlat
mov dl, al
mov ah, 02h
int 21h
```

```
mov al, ch
and al, 0Fh
xlat
mov dl, al
mov ah, 02h
int 21h
```

```
; Print Low Byte
mov al, cl    ; Low Byte
shr al, 4
and al, 0Fh
xlat
mov dl, al
mov ah, 02h
int 21h
```

```
mov al, cl
and al, 0Fh
xlat
mov dl, al
mov ah, 02h
int 21h
```

```
pop dx
pop cx
pop bx
pop ax
ret
```

```
; Subroutine: PrintBinary16
; Prints AX (16-bit value) In Binary Format
PrintBinary16:
```

```
push ax
push bx
push cx
push dx
```

```
mov bx, ax    ; Save Original Value
mov cx, 16    ; 16 Bits To Print
```

```

PrintBit16:
    rol bx, 1    ; Rotate Left Through Carry
    mov dl, '0'
    jnc PrintZero16
    mov dl, '1'
PrintZero16:
    push ax
    mov ah, 02h
    int 21h
    pop ax
    loop PrintBit16

    pop dx
    pop cx
    pop bx
    pop ax
    ret
; Subroutine: PrintDecimal16
; Prints AX (16-bit value) in Decimal Format (0-65535)
PrintDecimal16:
    push ax
    push bx
    push cx
    push dx

    mov bx, ax    ; Save Original Value
    mov ax, bx
    mov cx, 0     ; Initialize Digit Counter

DivideLoop:
    mov dx, 0
    mov bx, 10
    div bx        ; AX = Quotient, DX = Remainder
    push dx       ; Push Digit (0-9)
    inc cx
    cmp ax, 0
    jne DivideLoop

PrintLoop:
    pop dx
    add dl, '0'   ; Convert To ASCII
    mov ah, 02h
    int 21h
    loop PrintLoop

    pop dx
    pop cx
    pop bx
    pop ax
    ret

end main

```

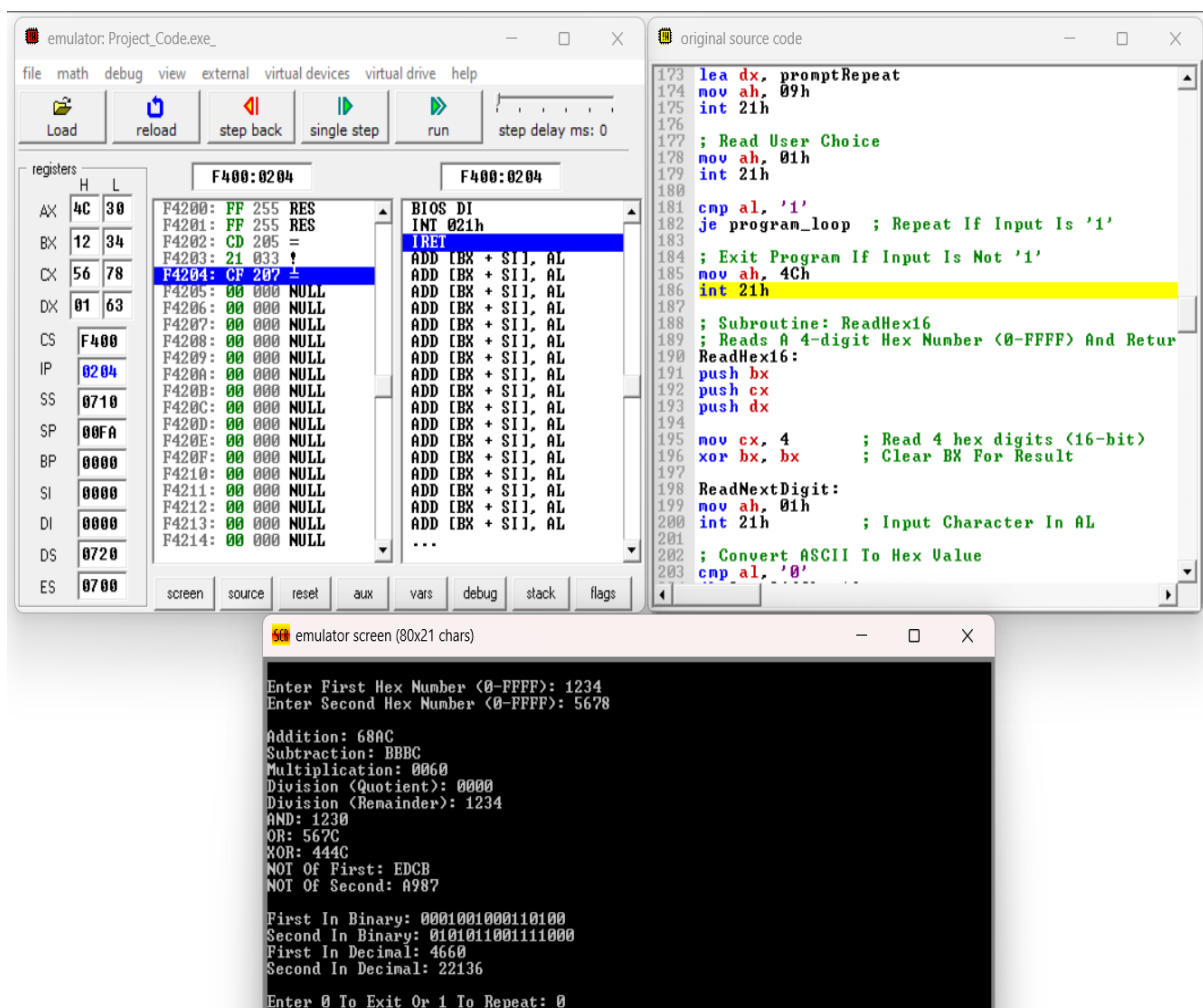
5. Results & Interpretation

Result -1 :

Let's Assume You Enter The Following Characters When Prompted:

- First Number: 1234 (In Hexadecimal)
- Second Number: 5678 (In Hexadecimal)
- Choice To Exit Or Continue: 0

Output :

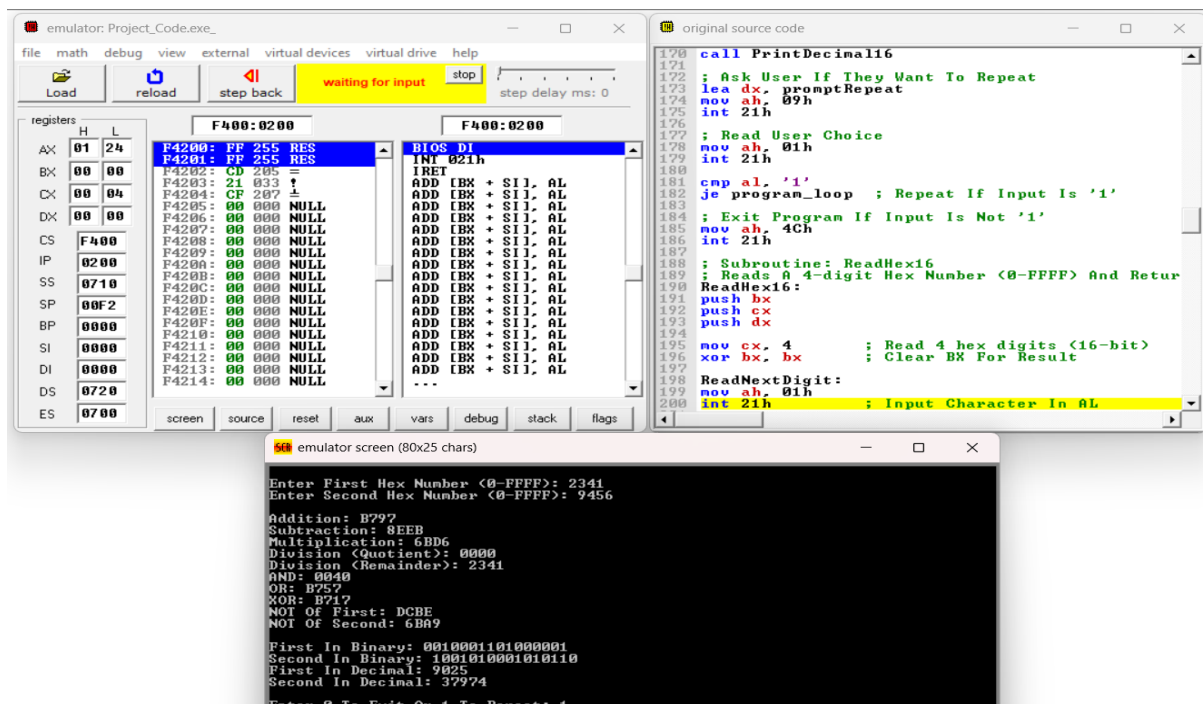


Result – 2 :

Let's Assume You Enter The Following Characters When Prompted:

- First Number: 2341 (In Hexadecimal)
- Second Number: 9456 (In Hexadecimal)
- Choice To Exit Or Continue: 1

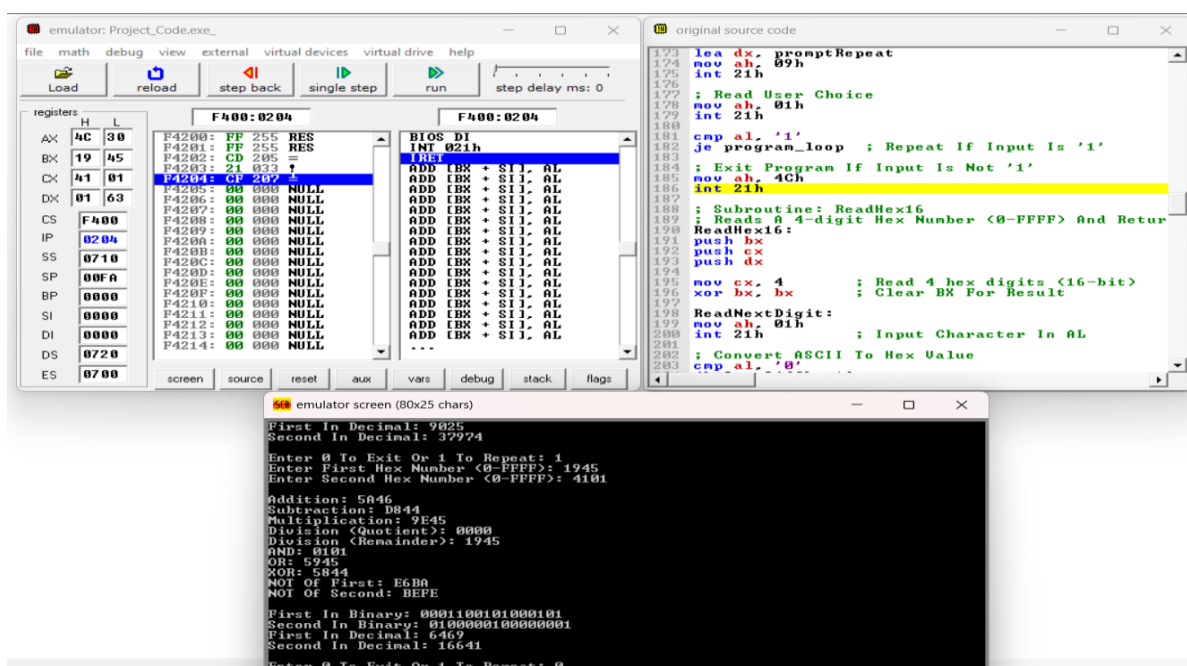
Output :



Let's Assume You Enter The Following Characters When Prompted:

- First Number: 1945 (In Hexadecimal)
- Second Number: 4101 (In Hexadecimal)
- Choice To Exit Or Continue: 0

Output :

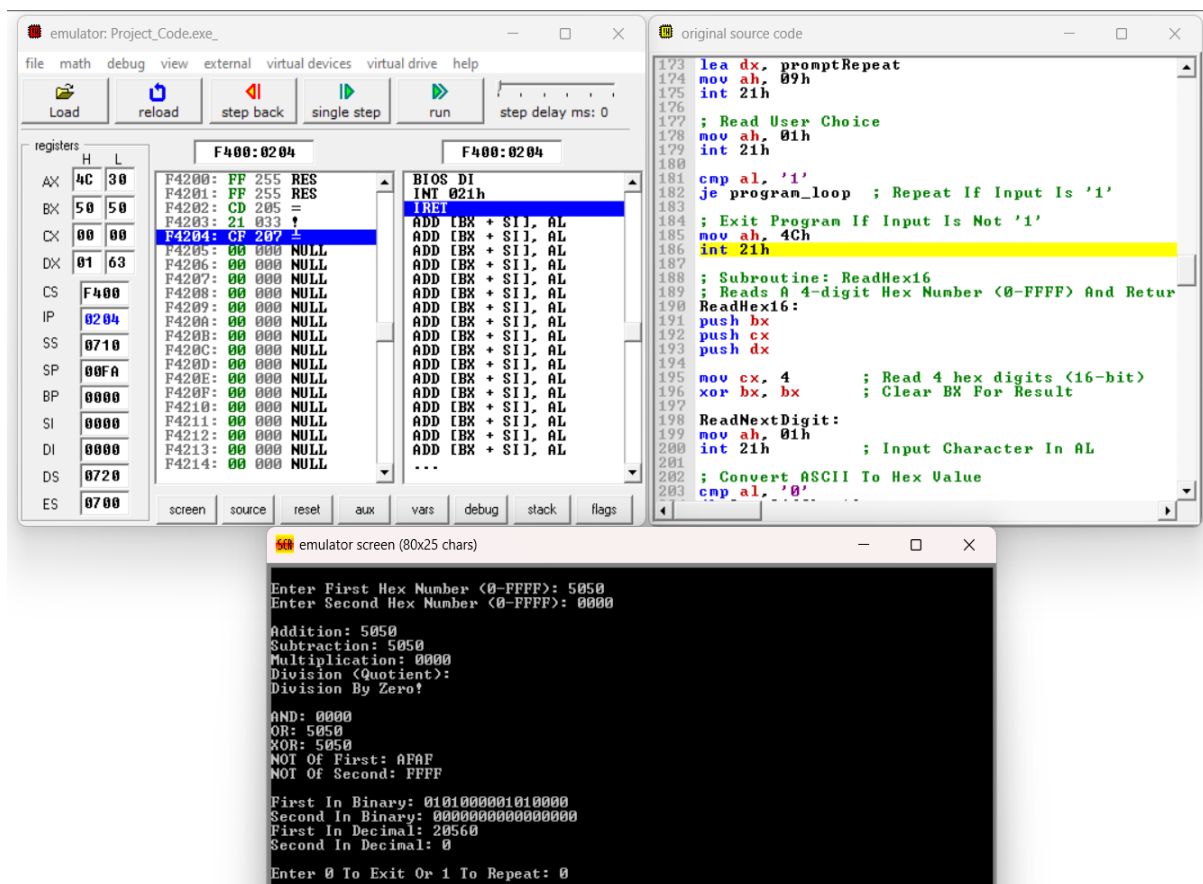


Result – 3 :

Let's Assume You Enter The Following Characters When Prompted:

- First Number: 5050 (In Hexadecimal)
- Second Number: 0000 (In Hexadecimal)
- Choice To Exit Or Continue: 0

Output :



6. Conclusion

The Design And Implementation Of An 16-Bit Calculator Using 8086 Assembly Language Effectively Demonstrate The Capabilities And Limitations Of Low-Level Programming For Arithmetic, Logical Operations, And Number System Conversions. This Project Highlights How Fundamental Computational Tasks Such As Addition, Subtraction, Bitwise Operations (AND, OR, XOR, NOT), And Conversions Between Binary, Decimal, And Hexadecimal Can Be Directly Controlled And Executed By The Processor. By Using Assembly Language, The Program Ensures Efficient Use Of CPU Registers And Memory, Giving Insight Into How Hardware-Level Operations Are Performed. The Project Also Reinforces The Understanding Of Interrupt-Driven Input/Output Through DOS Services, Allowing Interaction With Users Via Keyboard And Screen.

Despite Its Simplicity, This Calculator Serves As An Educational Tool To Appreciate The Complexity Behind High-Level Calculator Applications And Emphasizes The Importance Of Data Representation In Computer Systems. Handling Number System Conversions Within 8086 Assembly Deepens Comprehension Of Binary Arithmetic And Encoding, Which Is Critical For Embedded Systems And Hardware Design. Although Modern Systems Rely On Higher-Level Languages For Such Tasks, Mastering These Assembly Concepts Builds A Strong Foundation In Computer Architecture And System Programming. Overall, The Project Successfully Integrates Arithmetic, Logical, And Conversion Functions Into A Compact, Interactive Program, Showcasing The Practical Use Of 8086 Assembly In Real-World Applications.

References

(As Per The IEEE Recommendations)

- Intel Corporation, Intel® 64 And IA-32 Architectures Software Developer's Manual, Vol. 1, 2019.
- R. Hyde, The Art Of Assembly Language, 2nd Ed. San Francisco, CA, USA: No Starch Press, 2010.
- M. Beck, "8086 Assembly Language Programming Tutorial," 2021. Available.
- J. S. Irvine, Assembly Language For Intel-Based Computers, 6th Ed. Boston, MA, USA: Pearson, 2012.